3548. Equal Sum Grid Partition II

Solved

Hard | Topics | Companies | Hint

You are given an `m x n` matrix `grid` of positive integers. Your task is to determine if it is possible to make **either one horizontal or one vertical cut** on the grid such that:

- Each of the two resulting sections formed by the cut is **non-empty**.
- The sum of elements in both sections is **equal**, or can be made equal by discounting **at most** one single cell in total (from either section).
- If a cell is discounted, the rest of the section must **remain connected**.

Return `true` if such a partition exists; otherwise, return `false`.

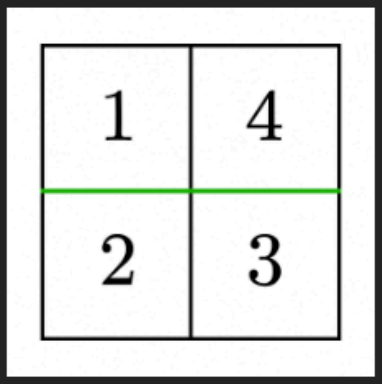
Note: A section is **connected** if every cell in it can be reached from any other cell by moving up, down, left, or right through other cells in the section.

Example 1:

Input: `grid = [[1,4],[2,3]]`

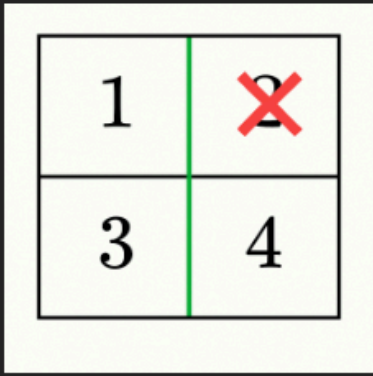
Output: `true`

Explanation:



- A horizontal cut after the first row gives sums `1 + 4 = 5` and `2 + 3 = 5`, which are equal. Thus, the answer is `true`.

Example 2:

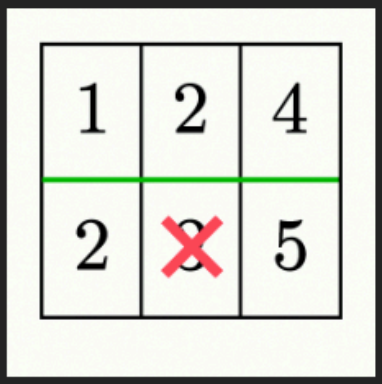
> **Input:** `grid = [[1,2],[3,4]]`
>
> **Output:** `true`
>
> **Explanation:**
>
> 
>
>
> - A vertical cut after the first column gives sums `1 + 3 = 4` and `2 + 4 = 6`.
> - By discounting 2 from the right section (`6 - 2 = 4`), both sections have equal sums and remain connected. Thus, the answer is `true`.

Example 3:

Input: `grid = [[1,2,4],[2,3,5]]`

Output: `false`

Explanation:



- A horizontal cut after the first row gives `1 + 2 + 4 = 7` and `2 + 3 + 5 = 10`.
- By discounting 3 from the bottom section (`10 - 3 = 7`), both sections have equal sums, but they do not remain connected as it splits the bottom section into two parts (`[2]` and `[5]`). Thus, the answer is `false`.

Example 4:

> **Input:** `grid = [[4,1,8],[3,2,6]]`
>
> **Output:** `false`
>
> **Explanation:**
>
> No valid cut exists, so the answer is `false`.

Constraints:

- `1 <= m == grid.length <= 10^5`
- `1 <= n == grid[i].length <= 10^5`
- `2 <= m * n <= 10^5`
- `1 <= grid[i][j] <= 10^5`

Python:

```
class Solution:
    def canPartitionGrid(self, grid):
        m, n = len(grid), len(grid[0])

        total = 0
        bottom = defaultdict(int)
        top = defaultdict(int)
        left = defaultdict(int)
        right = defaultdict(int)

        # Initialize bottom and right maps
        for row in grid:
            for x in row:
                total += x
                bottom[x] += 1
                right[x] += 1
```

```
sumTop = 0

# 🔹 Horizontal cuts
for i in range(m - 1):
    for j in range(n):
        val = grid[i][j]
        sumTop += val

        top[val] += 1
        bottom[val] -= 1

    sumBottom = total - sumTop

    if sumTop == sumBottom:
        return True

    diff = abs(sumTop - sumBottom)

    if sumTop > sumBottom:
        if self.check(top, grid, 0, i, 0, n - 1, diff):
            return True
    else:
        if self.check(bottom, grid, i + 1, m - 1, 0, n - 1, diff):
            return True

sumLeft = 0

# 🔹 Vertical cuts
for j in range(n - 1):
    for i in range(m):
        val = grid[i][j]
        sumLeft += val

        left[val] += 1
        right[val] -= 1

    sumRight = total - sumLeft

    if sumLeft == sumRight:
        return True

    diff = abs(sumLeft - sumRight)

    if sumLeft > sumRight:
```

```
                if self.check(left, grid, 0, m - 1, 0, j, diff):
                    return True
            else:
                if self.check(right, grid, 0, m - 1, j + 1, n - 1, diff):
                    return True

        return False

    def check(self, mp, grid, r1, r2, c1, c2, diff):
        rows = r2 - r1 + 1
        cols = c2 - c1 + 1

        # single cell
        if rows * cols == 1:
            return False

        # 1D row
        if rows == 1:
            return grid[r1][c1] == diff or grid[r1][c2] == diff

        # 1D column
        if cols == 1:
            return grid[r1][c1] == diff or grid[r2][c1] == diff

        return mp.get(diff, 0) > 0
```

JavaScript:

```
/**
 * @param {number[][]} grid
 * @return {boolean}
 */
var canPartitionGrid = function(grid) {
  const m = grid.length, n = grid[0].length;
  // row sums & col sums
  const rowSum = new Array(m).fill(0);
  const colSum = new Array(n).fill(0);
  for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
      rowSum[i] += grid[i][j];
      colSum[j] += grid[i][j];
    }
  }
  const totalSum = rowSum.reduce((a,b)=>a+b,0);
```

```
// build total frequency map
const totalCnt = new Map();
for (let i = 0; i < m; i++)
  for (let j = 0; j < n; j++)
    totalCnt.set(grid[i][j], (totalCnt.get(grid[i][j])||0) + 1);

// helper to check map has value
const hasInMap = (mp, v) => (mp.get(v) || 0) > 0;

// ---------- HORIZONTAL cuts ----------
// topCounts starts empty, bottomCounts = totalCnt
const topCounts = new Map();
const bottomCounts = new Map(totalCnt); // shallow copy of counts

let prefix = 0; // sumTop as we move cut down
for (let i = 1; i < m; i++) {
  // add row i-1 to topCounts, remove from bottomCounts
  for (let j = 0; j < n; j++) {
    const val = grid[i-1][j];
    topCounts.set(val, (topCounts.get(val)||0) + 1);
    bottomCounts.set(val, bottomCounts.get(val) - 1);
    if (bottomCounts.get(val) === 0) bottomCounts.delete(val);
  }
  prefix += rowSum[i-1];
  const sumTop = prefix;
  const sumBottom = totalSum - sumTop;
  if (sumTop === sumBottom) return true;
  const diff = Math.abs(sumTop - sumBottom);
  // decide which side is larger
  if (sumTop > sumBottom) {
    // need to find in top a removable cell == diff
    const topRows = i, topCols = n;
    // connectivity rule:
    if (topRows > 1 && topCols > 1) {
      if (hasInMap(topCounts, diff)) return true;
    } else if (topRows === 1) {
      // only endpoints allowed: row 0, cols 0 or n-1
      if (grid[0][0] === diff || grid[0][n-1] === diff) return true;
    } else if (topCols === 1) {
      // single column: allowed rows are 0 or i-1
      if (grid[0][0] === diff || grid[i-1][0] === diff) return true;
    }
  } else {
    // bottom larger: search in bottomCounts
```

```
    const bottomRows = m - i, bottomCols = n;
    if (bottomRows > 1 && bottomCols > 1) {
      if (hasInMap(bottomCounts, diff)) return true;
    } else if (bottomRows === 1) {
      // bottom is single row at index i
      if (grid[i][0] === diff || grid[i][n-1] === diff) return true;
    } else if (bottomCols === 1) {
      // single column: allowed rows are i or m-1 at col 0
      if (grid[i][0] === diff || grid[m-1][0] === diff) return true;
    }
  }
}

// ---------- VERTICAL cuts ----------
// leftCounts empty, rightCounts = totalCnt
const leftCounts = new Map();
const rightCounts = new Map(totalCnt);
let prefixCol = 0;
for (let j = 1; j < n; j++) {
  // add column j-1 to leftCounts, remove from rightCounts
  for (let i = 0; i < m; i++) {
    const val = grid[i][j-1];
    leftCounts.set(val, (leftCounts.get(val)||0) + 1);
    rightCounts.set(val, rightCounts.get(val) - 1);
    if (rightCounts.get(val) === 0) rightCounts.delete(val);
  }
  prefixCol += colSum[j-1];
  const sumLeft = prefixCol;
  const sumRight = totalSum - sumLeft;
  if (sumLeft === sumRight) return true;
  const diff = Math.abs(sumLeft - sumRight);
  if (sumLeft > sumRight) {
    // search in left
    const leftCols = j, leftRows = m;
    if (leftRows > 1 && leftCols > 1) {
      if (hasInMap(leftCounts, diff)) return true;
    } else if (leftCols === 1) {
      // left single column at col 0: allowed rows endpoints 0 or m-1
      if (grid[0][0] === diff || grid[m-1][0] === diff) return true;
    } else if (leftRows === 1) {
      // left is single row (m==1) at row 0: endpoints are col 0 or j-1
      if (grid[0][0] === diff || grid[0][j-1] === diff) return true;
    }
  } else {
```

```
      // right larger: search in rightCounts
      const rightCols = n - j, rightRows = m;
      if (rightRows > 1 && rightCols > 1) {
        if (hasInMap(rightCounts, diff)) return true;
      } else if (rightCols === 1) {
        // right single column at col j: allowed rows endpoints 0 or m-1
        if (grid[0][j] === diff || grid[m-1][j] === diff) return true;
      } else if (rightRows === 1) {
        // right single row (m==1) at row 0: endpoints are col j or n-1
        if (grid[0][j] === diff || grid[0][n-1] === diff) return true;
      }
    }
  }

  return false;
};
```

Java:

```
class Solution {
    public boolean canPartitionGrid(int[][] grid) {
        int m = grid.length, n = grid[0].length;

        long total = 0;

        HashMap<Long, Integer> bottom = new HashMap<>();
        HashMap<Long, Integer> top = new HashMap<>();
        HashMap<Long, Integer> left = new HashMap<>();
        HashMap<Long, Integer> right = new HashMap<>();

        // Initialize bottom and right maps
        for (int[] row : grid) {
            for (int x : row) {
                total += x;
                bottom.put((long)x, bottom.getOrDefault((long)x, 0) + 1);
                right.put((long)x, right.getOrDefault((long)x, 0) + 1);
            }
        }

        long sumTop = 0;

        // 🔹 Horizontal cuts
        for (int i = 0; i < m - 1; i++) {
            for (int j = 0; j < n; j++) {
                int val = grid[i][j];
```

```
            sumTop += val;

            top.put((long)val, top.getOrDefault((long)val, 0) + 1);
            bottom.put((long)val, bottom.get((long)val) - 1);
        }

        long sumBottom = total - sumTop;

        if (sumTop == sumBottom) return true;

        long diff = Math.abs(sumTop - sumBottom);

        if (sumTop > sumBottom) {
            if (check(top, grid, 0, i, 0, n - 1, diff)) return true;
        } else {
            if (check(bottom, grid, i + 1, m - 1, 0, n - 1, diff)) return true;
        }
    }

    long sumLeft = 0;

    // 🔹 Vertical cuts
    for (int j = 0; j < n - 1; j++) {
        for (int i = 0; i < m; i++) {
            int val = grid[i][j];
            sumLeft += val;

            left.put((long)val, left.getOrDefault((long)val, 0) + 1);
            right.put((long)val, right.get((long)val) - 1);
        }

        long sumRight = total - sumLeft;

        if (sumLeft == sumRight) return true;

        long diff = Math.abs(sumLeft - sumRight);

        if (sumLeft > sumRight) {
            if (check(left, grid, 0, m - 1, 0, j, diff)) return true;
        } else {
            if (check(right, grid, 0, m - 1, j + 1, n - 1, diff)) return true;
        }
    }
```

```
        return false;
    }

    private boolean check(HashMap<Long, Integer> mp, int[][] grid,
                          int r1, int r2, int c1, int c2, long diff) {

        int rows = r2 - r1 + 1;
        int cols = c2 - c1 + 1;

        // single cell
        if (rows * cols == 1) return false;

        // 1D row
        if (rows == 1) {
            return grid[r1][c1] == diff || grid[r1][c2] == diff;
        }

        // 1D column
        if (cols == 1) {
            return grid[r1][c1] == diff || grid[r2][c1] == diff;
        }

        return mp.getOrDefault(diff, 0) > 0;
    }
}
```